

0.1 Thiết kế kiến trúc

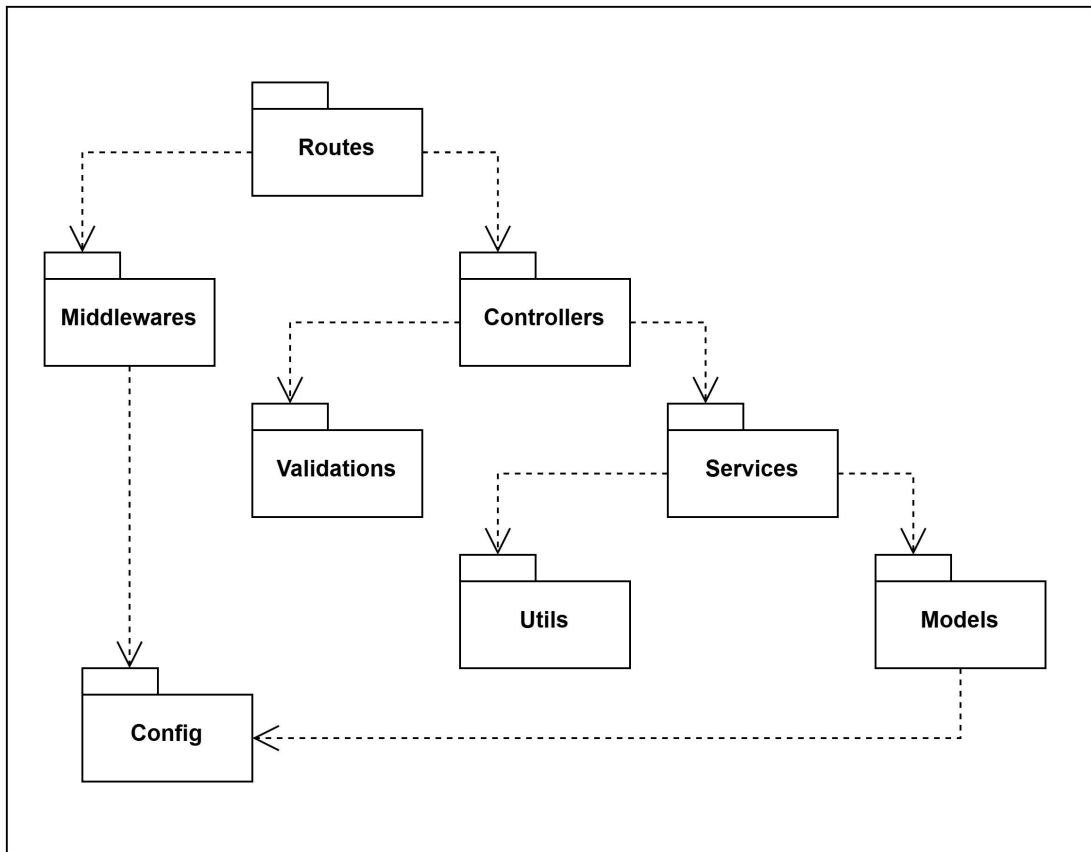
0.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống Quản lý Học viên được xây dựng dựa trên kiến trúc phân lớp (Layered Architecture) trong môi trường Client-Server hiện đại. Hệ thống được tổ chức thành các tầng độc lập xếp chồng lên nhau, giao tiếp tuần tự từ trên xuống dưới để đảm bảo tính phân tách mối quan tâm (Separation of Concerns), giúp mã nguồn dễ bảo trì và nâng cấp độc lập:

- **Tầng Giao diện (Presentation Layer):** Đảm nhận việc hiển thị thông tin trực quan và tiếp nhận các hành động tương tác từ người dùng (Học viên, Cán bộ chỉ huy, Quản trị viên). Giao diện được xây dựng phía Client sử dụng Next.js kết hợp Tailwind CSS với các thành phần giao diện điển hình như `UserProfile` (thông tin cá nhân, kết quả rèn luyện), `GradeManagement` (xem bảng điểm, đề xuất cập nhật điểm số), `CutRiceRegistration` (đăng ký cắt cơm), `TuitionManagement` (theo dõi học phí) và `Dashboard` (biểu đồ thống kê học tập).
- **Tầng Điều phối và Kiểm soát (Controller Layer):** Đóng vai trò cửa ngõ tiếp nhận các yêu cầu HTTP API gửi từ Client lên Server. Controller chịu trách nhiệm giải nén tham số, thực hiện lọc bảo mật (JWT middleware), kiểm tra tính hợp lệ của dữ liệu đầu vào và chuyển tiếp yêu cầu đến tầng nghiệp vụ. Trong hệ thống, tầng này sử dụng các lớp như `auth.controller`, `user.controller`, `gradeRequest.controller`, `cutRice.controller` và `tuitionFee.controller`.
- **Tầng Nghiệp vụ (Service Layer):** Là trung tâm xử lý logic cốt lõi và các quy tắc nghiệp vụ của hệ thống. Tầng này thực thi các quy trình tính toán phức tạp như tự động tính toán lại điểm trung bình GPA/CPA hệ 4 và hệ 10 khi có thay đổi điểm số, tự động lập lịch cắt cơm dựa trên thời khóa biểu, hoặc tính toán công nợ học phí. Tầng này hiện thực thông qua các lớp dịch vụ như `student.service`, `gradeRequest.service`, `cutRice.service`, `tuitionFee.service` và `yearlyResult.service`.
- **Tầng Truy xuất Dữ liệu (Model Layer):** Chịu trách nhiệm lưu trữ, truy xuất và cập nhật thông tin trong cơ sở dữ liệu quan hệ PostgreSQL thông qua Sequelize ORM. Tầng này thiết lập các ràng buộc toàn vẹn dữ liệu, các mối quan hệ khóa ngoại và cung cấp các phương thức thao tác dữ liệu trừu tượng cho tầng nghiệp vụ sử dụng, thông qua các lớp thực thể như `user.model`, `profile.model`, `subjectResult.model`, `gradeRequest.model`, `cutRice.model`, `tuitionFee.model` và `yearlyResult.model`.

0.1.2 Thiết kế tổng quan

a, Biểu đồ phụ thuộc gói Server



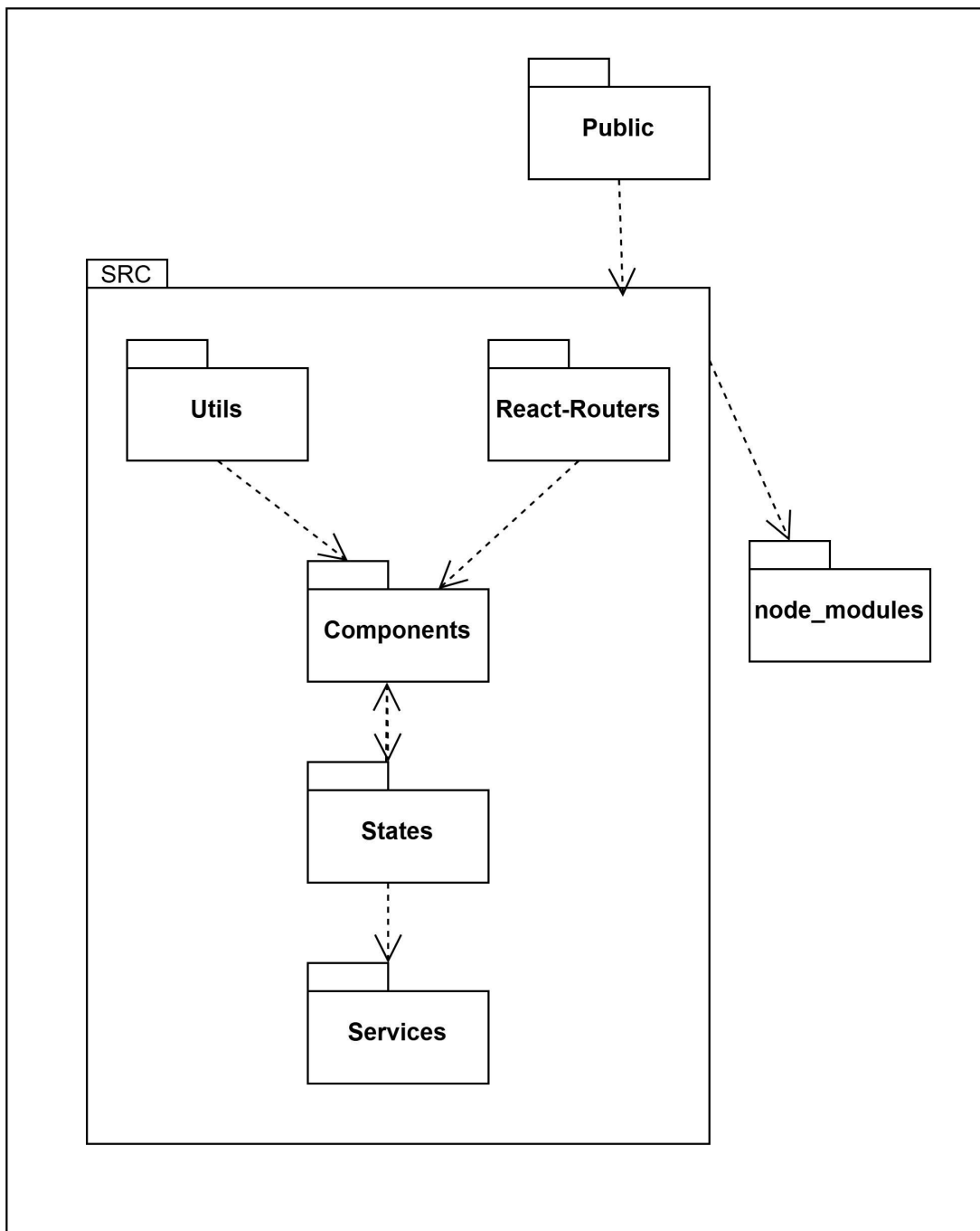
Hình 0.1: Biểu đồ gói cho sever

Hệ thống server được thiết kế theo mô hình phân lớp nghiêm ngặt nhằm đảm bảo tính rõ ràng, dễ bảo trì và mở rộng. Các request từ client trước tiên được xử lý tại **Routes**, nơi định tuyến đến các **Controllers** phù hợp dựa trên đường dẫn và phương thức HTTP. **Controllers** đóng vai trò điều phối, chịu trách nhiệm nhận dữ liệu yêu cầu, gọi **Validations** để kiểm tra tính hợp lệ của dữ liệu đầu vào và chuyển giao xử lý cho lớp **Services**. **Middlewares** đảm nhiệm các chức năng trung gian xuyên suốt hệ thống như xác thực người dùng (Authentication), kiểm tra phân quyền (Authorization) và xử lý lỗi tập trung. **Services** chịu trách nhiệm thực hiện toàn bộ logic nghiệp vụ lõi (như tính toán điểm CPA, tự động lập lịch cắt cơm) và sẽ tương tác với lớp **Models** khi cần cập nhật dữ liệu, hoặc sử dụng các thư viện hỗ trợ tại **Utils**. **Models** định nghĩa các thực thể dữ liệu (như học viên, điểm số, lịch cơm, học phí...) và làm việc trực tiếp với Cơ sở dữ liệu thông qua cấu hình tại **Config**. Toàn bộ hoạt động của hệ thống được hỗ trợ bởi các thư viện bên ngoài được quản lý trong **Node Modules**.

Mục đích và sự phụ thuộc chi tiết của từng gói thành phần:

- **Routes (routes):** Tiếp nhận các yêu cầu HTTP từ Client và định tuyến đến Controller thích hợp. Gói này phụ thuộc trực tiếp vào **Controllers** (để chuyển tiếp xử lý) và **Middlewares** (để thực hiện các bộ lọc bảo mật trước khi vào controller).
- **Middlewares (middlewares):** Xử lý các tác vụ trung gian xuyên suốt hệ thống như xác thực token JWT, kiểm tra quyền hạn người dùng (Học viên, Chỉ huy, Admin) và bắt lỗi hệ thống. Gói này phụ thuộc vào **Config** để lấy thông tin khóa bảo mật JWT.
- **Controllers (controllers):** Điều phối luồng xử lý của từng API. Gói này phụ thuộc vào **Validations** (để xác thực cấu trúc dữ liệu đầu vào) và **Services** (để thực thi các nghiệp vụ tương ứng).
- **Validations (validations):** Định nghĩa các lược đồ xác thực dữ liệu đầu vào (Yup Schema) nhằm ngăn chặn dữ liệu sai cấu trúc hoặc mã độc từ Client trước khi gửi xuống lớp sâu hơn.
- **Services (services):** Nơi chứa toàn bộ logic xử lý nghiệp vụ chính của hệ thống. Gói này phụ thuộc trực tiếp vào **Models** (để truy vấn/ghi dữ liệu vào CSDL) và **Utils** (để sử dụng các hàm tiện ích trợ giúp).
- **Models (models):** Khởi tạo thực thể dữ liệu dưới dạng Sequelize Models ánh xạ trực tiếp đến PostgreSQL. Gói này phụ thuộc vào **Config** để thiết lập và duy trì kết nối cơ sở dữ liệu.
- **Config (config):** Quản lý cấu hình toàn hệ thống bao gồm kết nối PostgreSQL, kết nối máy chủ lưu trữ MinIO và các tham số môi trường khác.
- **Utils (utils):** Cung cấp các hàm hỗ trợ dùng chung trong hệ thống như công cụ tính toán điểm trung bình, định dạng ngày tháng, xuất báo cáo PDF/Excel.

b, Biểu đồ phụ thuộc gói Client



Hình 0.2: Biểu đồ gói cho client

Hình 4.3 mô tả hệ thống Client được xây dựng dựa trên kiến trúc thành phần của Next.js (React), trong đó mã nguồn chính nằm trong thư mục `src`. Cấu trúc được chia thành các module rõ ràng nhằm tối ưu hóa khả năng mở rộng và bảo trì. Cụ thể:

- **React-Routers (Định tuyến):** Đảm nhiệm việc định tuyến URL và điều hướng luồng hiển thị trang ứng dụng (trong thực tế dự án được triển khai bằng cơ chế

Next.js App Router). Gói này phụ thuộc trực tiếp vào **Components** để render các thành phần giao diện tương ứng với các trang.

- **Components (Thành phần giao diện):** Chứa các khối giao diện người dùng có thể tái sử dụng (như Button, Table, Modal, Sidebar...). Lớp này có mối quan hệ phụ thuộc hai chiều với **States** để cập nhật trạng thái hiển thị giao diện và thu thập dữ liệu đầu vào.
- **States (Quản lý trạng thái):** Quản lý trạng thái toàn cục của ứng dụng để đồng bộ dữ liệu giữa các thành phần giao diện (trong thực tế dự án sử dụng thư viện Zustand). **States** phụ thuộc trực tiếp vào **Services** để gọi API đồng bộ dữ liệu từ phía máy chủ.
- **Services (Dịch vụ kết nối):** Cung cấp các chức năng gọi API (thông qua Axios/React Query) và xử lý logic kết nối trực tiếp với backend Express.js.
- **Utils (Tiện ích):** Tập hợp các hàm tiện ích hỗ trợ định dạng dữ liệu, chuyển đổi hệ đếm hoặc các xử lý lặp lại. Gói này được import trực tiếp bởi **Components** để định dạng dữ liệu trước khi hiển thị.

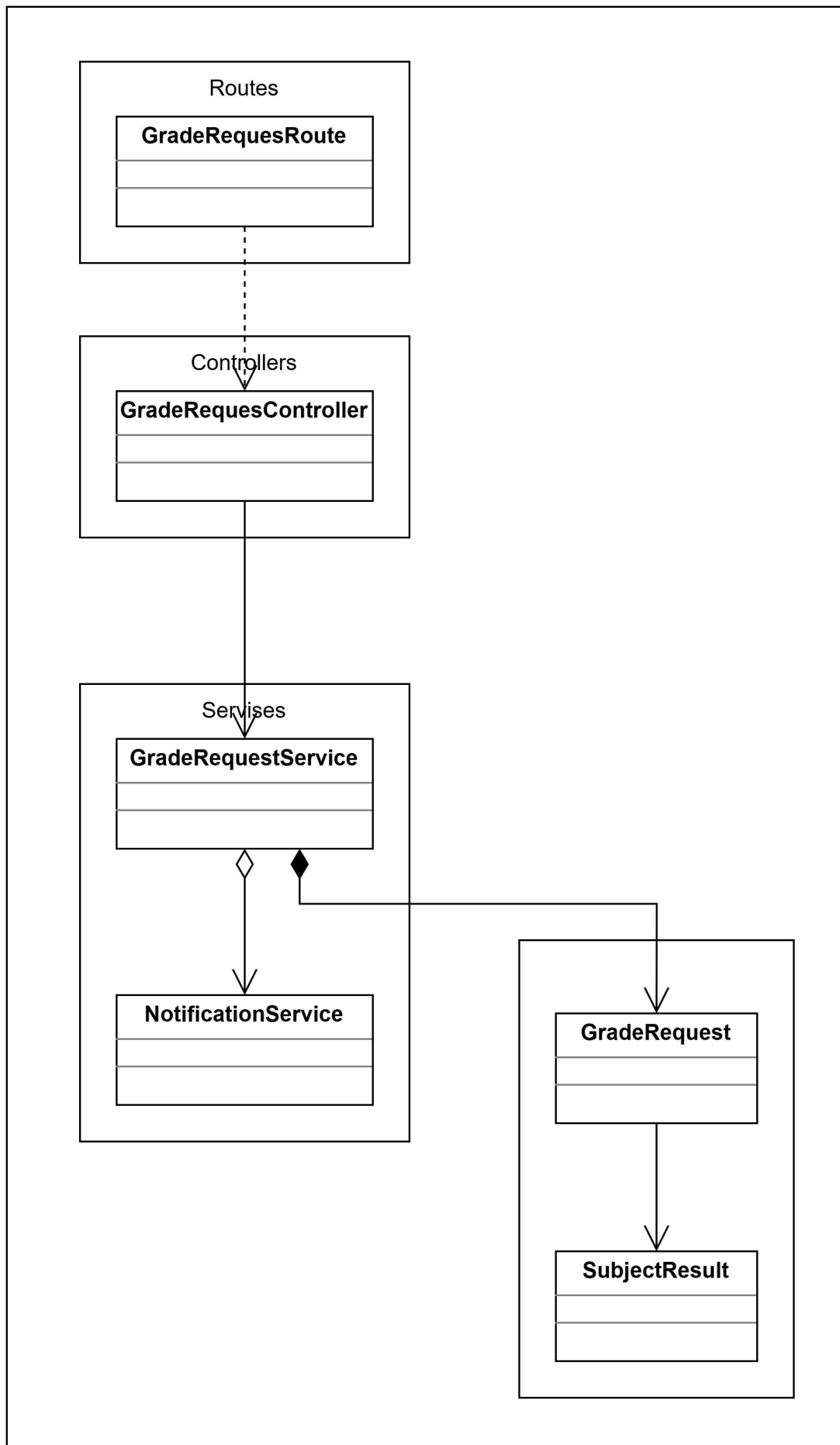
Ngoài ra, các thư mục **node-modules** và **public** lần lượt chứa các gói thư viện phụ thuộc bên ngoài và tài nguyên tĩnh (như hình ảnh, biểu tượng) phục vụ quá trình xây dựng và vận hành ứng dụng.

0.1.3 Thiết kế chi tiết gói

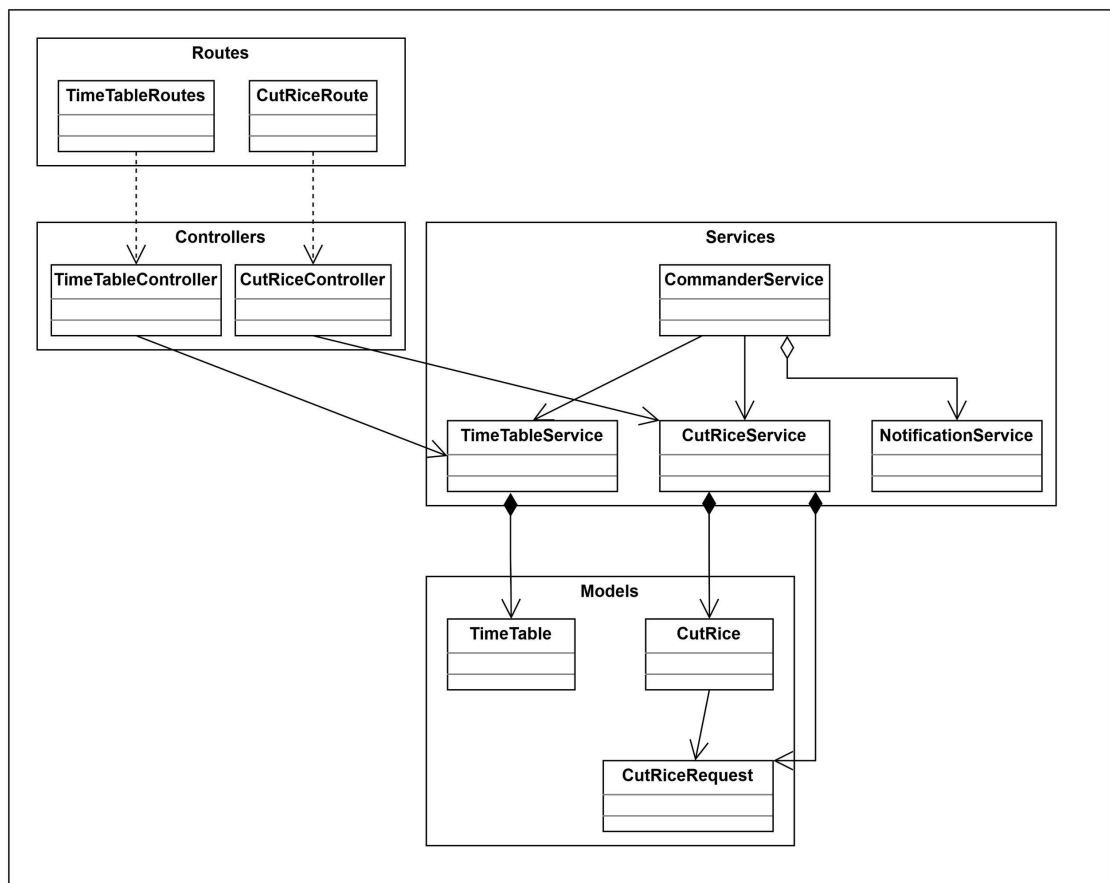
Sinh viên thiết kế và lần lượt vẽ biểu đồ thiết kế cho từng package, hoặc một nhóm các package liên quan để giải quyết một vấn đề gì đó. Khi vẽ thiết kế gói, sinh viên chỉ cần đưa tên lớp, không cần chỉ ra các thành viên phương thức và thuộc tính. SV tham khảo ví dụ minh họa trong Hình 0.15.

Sinh viên cần vẽ rõ ràng quan hệ giữa các lớp trong biểu đồ. Các quan hệ bao gồm: phụ thuộc (dependency), kết hợp (association), kết tập (aggregation), hợp thành (composition), kế thừa (inheritance), và thực thi (implementation). Các quan hệ này đều đã được minh họa trong 0.15.

Sau khi vẽ hình minh họa, sinh viên cần giải thích ngắn gọn về thiết kế của mình.



Hình 0.3: Thiết kế chi tiết các mối quan hệ lớp trong phân hệ Đề xuất và phê duyệt điểm



Hình 0.4: Thiết kế chi tiết các mối quan hệ lớp trong phân hệ Cắt cơm

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Hệ thống xây dựng giao diện trực quan, dễ sử dụng, dễ thao tác và nhất quán, thân thiện với người dùng.

Màu sắc Màu sắc ở hình ?? là bộ màu chính được sử dụng xuyên suốt trong hệ thống, đóng vai trò là màu chủ đạo để xây dựng nền, nút bấm chính, các thông báo quan trọng, các ô nhập dữ liệu và các thành phần giao diện khác.

- **Màu đậm nhất:** #313324
- **Màu nhạt nhất:** #F1F2EE



Hình 0.5: Bảng màu chính của hệ thống

Màu sắc ở hình ?? là nhóm màu phụ trợ, được sử dụng nhằm hỗ trợ bộ màu chính trong việc tạo điểm nhấn cho giao diện. Nhóm màu này thường được áp dụng cho

các nút bấm phụ hoặc các thành phần giao diện không yêu cầu mức độ chú ý cao từ người dùng.

- **Màu đậm nhất:** #31332A
- **Màu nhạt nhất:** #F1F2F0



Hình 0.6: Bảng màu phụ trợ của hệ thống

Màu sắc ở hình ?? là nhóm màu được sử dụng riêng cho các trạng thái cảnh báo, thông báo lỗi hệ thống, thao tác xóa dữ liệu hoặc các thông báo yêu cầu người dùng chú ý đặc biệt.

- **Màu đậm nhất:** #5D1815
- **Màu nhạt nhất:** #FCEBEA



Hình 0.7: Bảng màu trạng thái lỗi của hệ thống

Chữ viết Hình ?? đại diện cho phong chữ UI Sans Serif.

Ui Sans Serif Light
Ui Sans Serif Regular
Ui Sans Serif Semibold
Ui Sans Serif Bold
Ui Sans Serif Extrabold

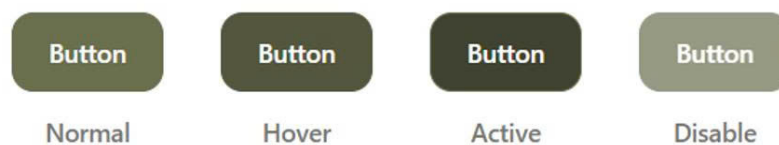
Hình 0.8: Phong chữ UI Sans Serif

Kiểu chữ xuyên suốt toàn bộ hệ thống được sử dụng là kiểu chữ Ui Sans Serif. UI Sans Serif là nhóm phong chữ sans-serif do hệ điều hành và trình duyệt cung cấp, được thiết kế và tối ưu riêng cho các thành phần giao diện người dùng. Khi sử dụng UI Sans Serif, hệ thống sẽ tự động lựa chọn phong chữ phù hợp với từng

nền tảng, chẳng hạn như San Francisco trên macOS, Segoe UI trên Windows hoặc Roboto trên Android. Cách tiếp cận này giúp giao diện đạt được sự nhất quán về khả năng hiển thị, đảm bảo tính dễ đọc và tạo cảm giác quen thuộc đối với người dùng trên nhiều thiết bị khác nhau. Đồng thời, việc sử dụng phông chữ hệ thống còn góp phần cải thiện hiệu năng tải trang, do không phát sinh chi phí tải font bên ngoài, từ đó nâng cao trải nghiệm người dùng tổng thể.

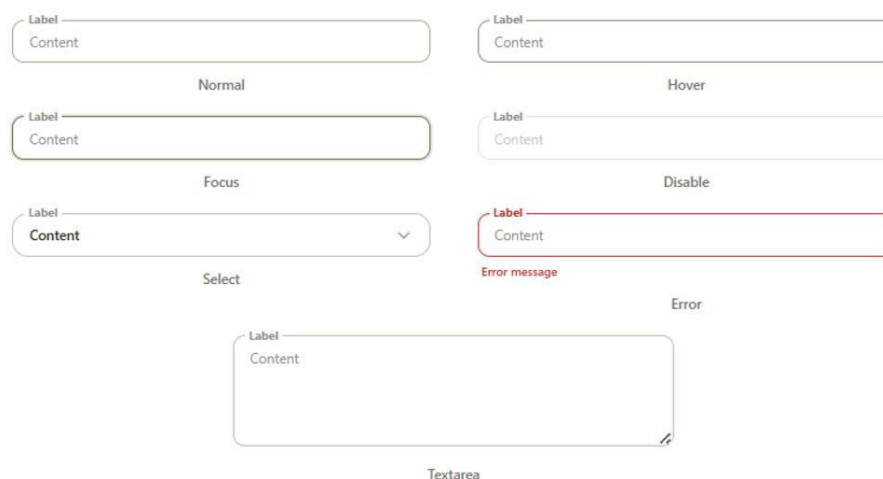
Cỡ chữ: Hệ thống hiển thị giao diện với đa dạng cỡ chữ: 12px, 14px, 16px, 18px, 20px, 24px, 30px, 36px, 48px, 60px, 72px, 96px, 128px.

Nút bấm (Button) Các nút bấm cơ bản được thiết kế như ở hình ???. Chúng được xây dựng phù hợp với bộ màu của toàn hệ thống nhằm đảm bảo tính nhất quán trong giao diện và giúp người dùng dễ dàng nhận biết chức năng của từng nút.

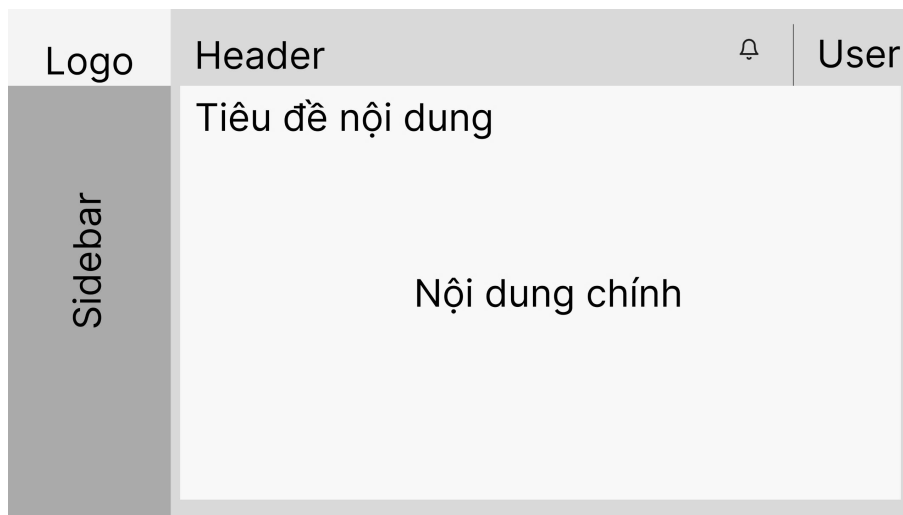


Hình 0.9: Các nút bấm cơ bản của hệ thống

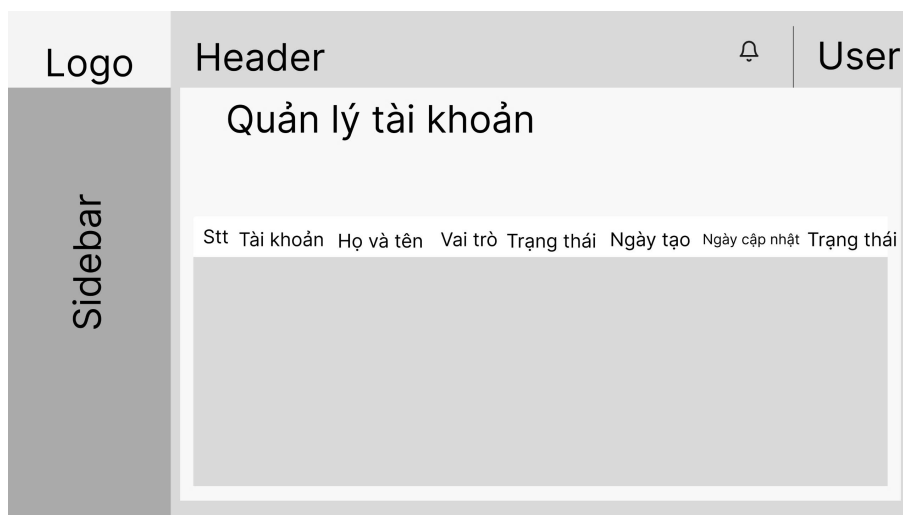
Ô nhập liệu (Input) Các ô nhập liệu cơ bản được thiết kế như ở hình ???. Chúng được xây dựng phù hợp với giao diện tổng thể của hệ thống, đồng thời thể hiện rõ các trạng thái trong quá trình nhập dữ liệu như trạng thái mặc định, đang nhập, hợp lệ, không hợp lệ hoặc vô hiệu hóa.



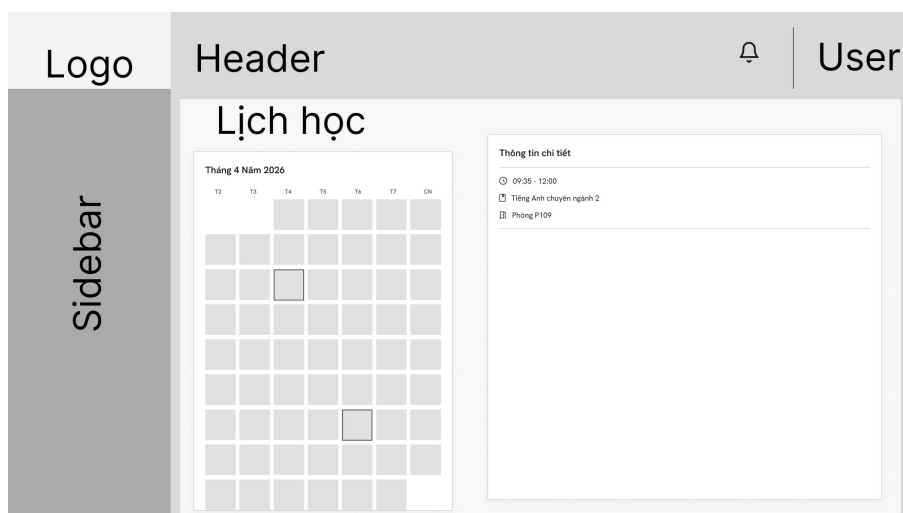
Hình 0.10: Các ô nhập liệu cơ bản của hệ thống



Hình 0.11: Thiết kế giao diện bố cục chung



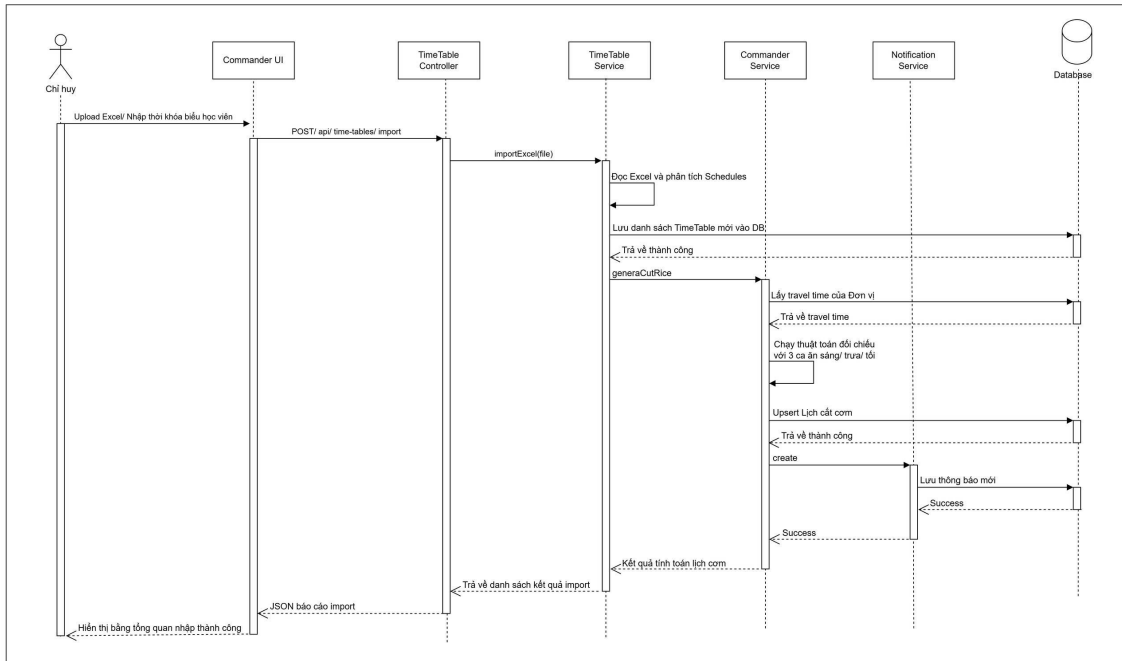
Hình 0.12: Thiết kế giao diện quản lý tài khoản



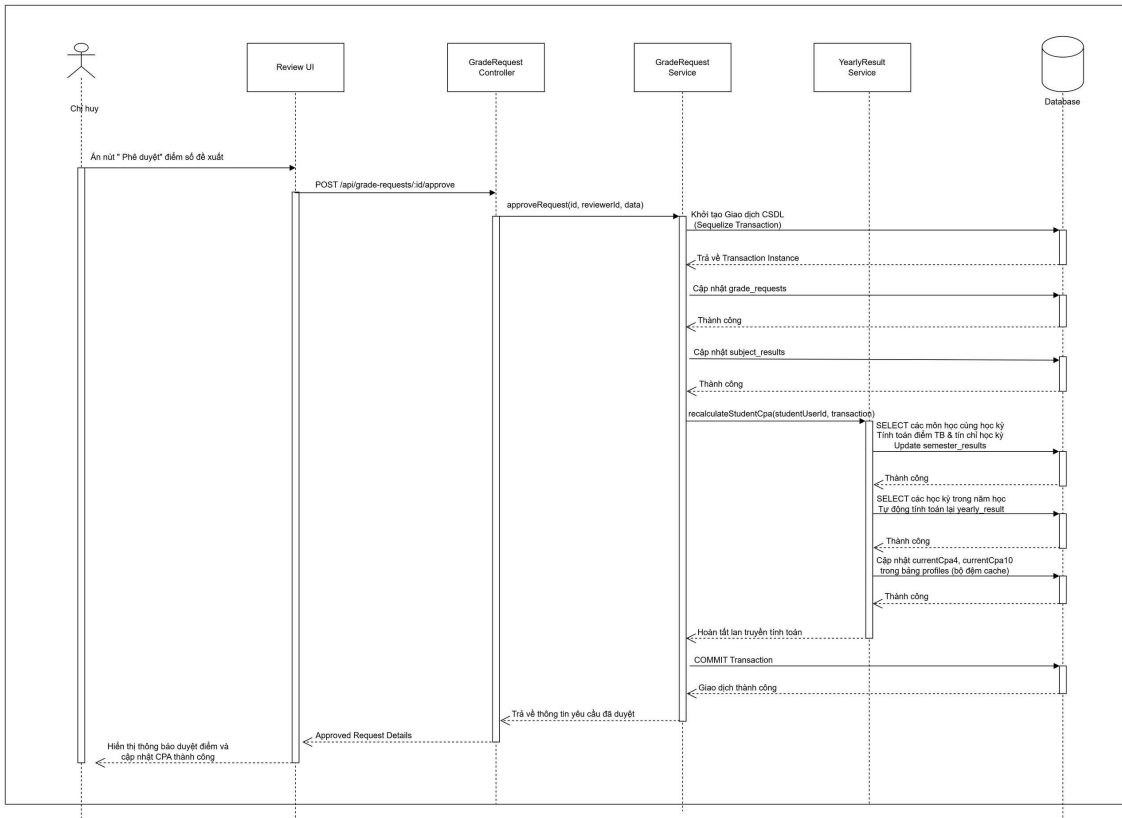
Hình 0.13: Thiết kế giao diện quản lý lịch học

0.2.2 Thiết kế lớp

a, Biểu đồ tuần tự cập nhật lịch cắt cơm từ thời khóa biểu

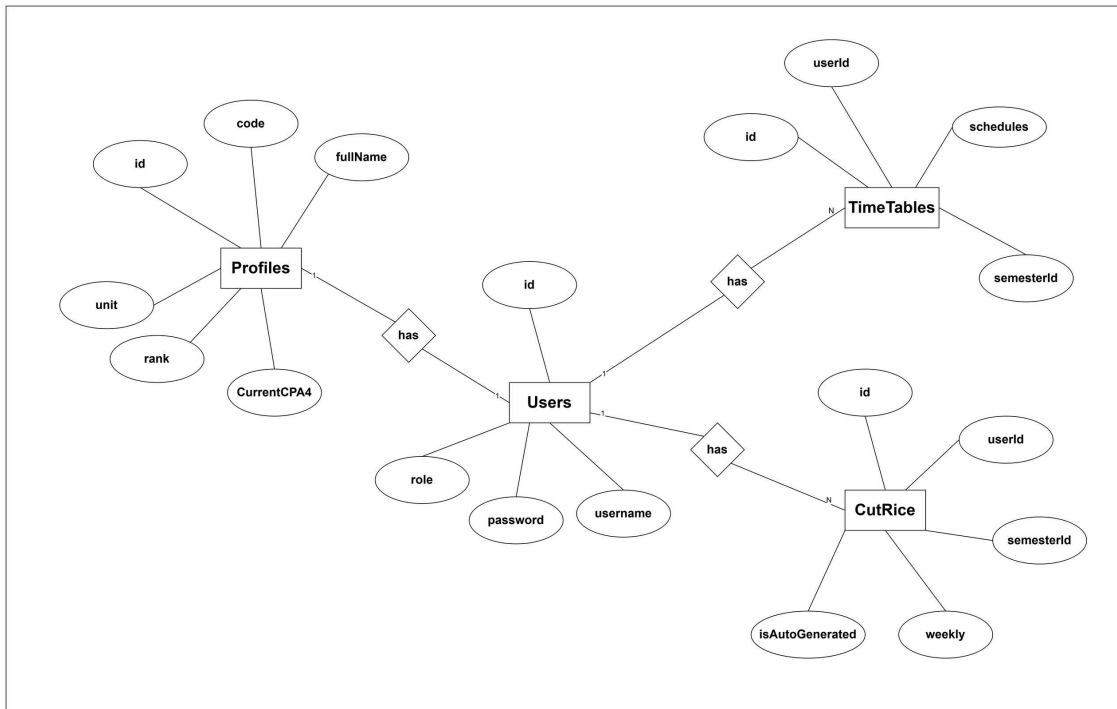


b, Biểu đồ tuần tự phê duyệt điểm và tính toán điểm CPA



0.2.3 Thiết kế cơ sở dữ liệu

a, Biểu đồ thực thể liên kết (E-R diagram)



Hình 0.14: Biểu đồ thực thể liên kết (E-R diagram)

Thực thể Users (Tài khoản)

- **Các thuộc tính chính:** id, username, password, is_admin, role, profile_id.
- **Mô tả:** Lưu trữ thông tin tài khoản đăng nhập và quyền hạn của người dùng (Học viên, Chỉ huy, Admin) trong hệ thống.
- **Mối quan hệ:** Mỗi tài khoản liên kết trực tiếp với duy nhất một hồ sơ quân nhân cá nhân (profile_id là khóa ngoại trỏ tới bảng profiles). Một tài khoản học viên có thể có nhiều lịch học và lịch cắt cõm (id là khóa ngoại trong các bảng time_tables và cut_rice).

Thực thể Profiles (Hồ sơ)

- **Các thuộc tính chính:** id, code, fullName, gender, birthday, unit, rank, position_government, currentCpa4, currentCpa10, commander_id.
- **Mô tả:** Lưu trữ thông tin lý lịch cá nhân chi tiết, quân tịch và học lực của cả học viên và cán bộ chỉ huy.

- **Mối quan hệ:** Mỗi hồ sơ sở hữu duy nhất một tài khoản đăng nhập (`id` là khóa ngoại trỏ tới bảng `users`). Mỗi hồ sơ học viên được quản lý trực tiếp bởi một chỉ huy (`commander_id` là khóa ngoại tự liên kết đến bảng `profiles`).

Thực thể TimeTables (Thời khóa biểu)

- **Các thuộc tính chính:** `id`, `userId`, `semester_id`, `schedules`.
- **Mô tả:** Lưu trữ thông tin thời khóa biểu học tập của học viên theo từng học kỳ dưới dạng danh sách các ca học.
- **Mối quan hệ:** Mỗi thời khóa biểu thuộc về một tài khoản học viên (`userId` là khóa ngoại trỏ tới bảng `users`).

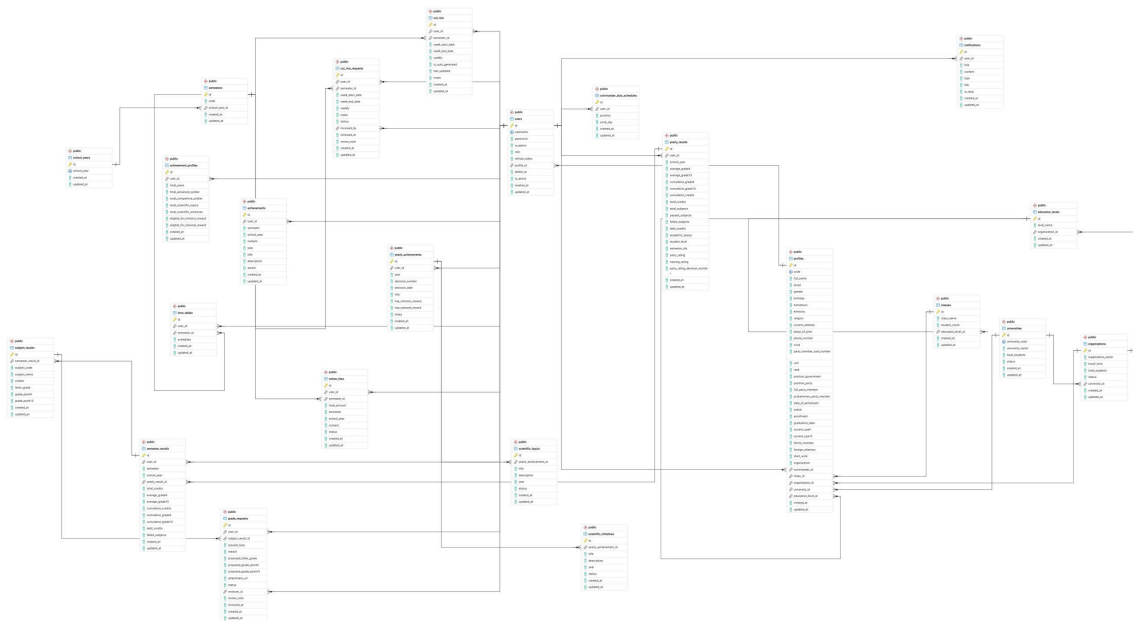
Thực thể CutRice (Lịch cắt cơm)

- **Các thuộc tính chính:** `id`, `userId`, `semester_id`, `weekly`, `isAutoGenerated`, `notes`.
- **Mô tả:** Quản lý việc báo cắt cơm ăn của học viên theo tuần (Sáng, Trưa, Tối) nhằm tối ưu hóa hậu cần và tránh lãng phí quân lương.
- **Mối quan hệ:** Mỗi lịch cắt cơm thuộc về một tài khoản học viên (`userId` là khóa ngoại trỏ tới bảng `users`).

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
<code>id</code>	<code>uuid</code>	Khóa chính của lịch cắt cơm
<code>userId</code>	<code>uuid</code>	Khóa ngoại liên kết tới tài khoản học viên
<code>semesterId</code>	<code>uuid</code>	Khóa ngoại liên kết tới học kỳ
<code>weekStartDate</code>	<code>date</code>	Ngày bắt đầu của tuần cắt cơm
<code>weekEndDate</code>	<code>date</code>	Ngày kết thúc của tuần cắt cơm
<code>weekly</code>	<code>jsonb</code>	Trạng thái cắt cơm chi tiết theo tuần (Sáng/Trưa/Tối)
<code>isAutoGenerated</code>	<code>boolean</code>	Trạng thái tự động sinh từ thời khóa biểu
<code>lastUpdated</code>	<code>timestamp</code>	Thời gian cập nhật gần nhất
<code>notes</code>	<code>varchar(255)</code>	Ghi chú thêm

Bảng 1: Bảng miêu tả chi tiết bảng `cut_rice`

b, Thiết kế cơ sở dữ liệu



Hình 0.15: Cơ sở dữ liệu

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	serial	Id của người dùng
username	varchar(255)	Tên đăng nhập của người dùng
password	varchar(255)	Mật khẩu người dùng
name	varchar(255)	Họ tên người dùng
dob	timestamp with time zone	Ngày sinh của người dùng
resetOTP	varchar(255)	Mã OTP dùng để khôi phục tài khoản
phone	varchar(255)	Số điện thoại
email	varchar(255)	Email người dùng
balance	double precision	Số dư tài khoản
role	enum_users_role	Vai trò của người dùng
avatar	varchar(255)	Đường dẫn avatar
gender	enum_users_gender	Giới tính
resetOTPExpiry	timestamp with time zone	Thời gian hết hạn mã OTP

Bảng 2: Bảng miêu tả chi tiết bảng user

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	uuid	Khóa chính của hồ sơ
code	varchar(50)	Mã học viên / quân nhân
fullName	varchar(100)	Họ và tên
email	varchar(100)	Địa chỉ email
gender	varchar(20)	Giới tính
birthday	timestamp	Ngày sinh
hometown	varchar(255)	Quê quán
phoneNumber	varchar(20)	Số điện thoại
cccd	varchar(20)	Số Căn cước công dân
unit	varchar(255)	Đơn vị trực thuộc
rank	varchar(50)	Cấp bậc / Quân hàm
positionGovernment	varchar(100)	Chức vụ chính quyền
currentCpa4	double precision	Điểm CPA tích lũy hệ 4
currentCpa10	double precision	Điểm CPA tích lũy hệ 10
commanderId	uuid	Khóa ngoại liên kết tới cán bộ quản lý

Bảng 3: Bảng miêu tả chi tiết bảng profiles

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	uuid	Khóa chính kết quả môn học
semesterResultId	uuid	Khóa ngoại liên kết kết quả học kỳ
subjectCode	varchar(50)	Mã môn học
subjectName	varchar(255)	Tên môn học
credits	integer	Số tín chỉ môn học
letterGrade	varchar(5)	Điểm chữ (A, B, C, D, F)
gradePoint4	double precision	Điểm môn học hệ 4
gradePoint10	double precision	Điểm môn học hệ 10

Bảng 4: Bảng miêu tả chi tiết bảng subject_results

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	uuid	Khóa chính của lịch cắt cõm
userId	uuid	Khóa ngoại liên kết tới tài khoản học viên
semesterId	uuid	Khóa ngoại liên kết tới học kỳ
weekStartDate	date	Ngày bắt đầu của tuần cắt cõm
weekEndDate	date	Ngày kết thúc của tuần cắt cõm
weekly	jsonb	Trạng thái cắt cõm chi tiết theo tuần (Sáng/Trưa/Tối)
isAutoGenerated	boolean	Trạng thái tự động sinh từ thời khóa biểu
lastUpdated	timestamp	Thời gian cập nhật gần nhất
notes	varchar(255)	Ghi chú thêm

Bảng 5: Bảng miêu tả chi tiết bảng cut_rice

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	uuid	Khóa chính của bản ghi học phí
userId	uuid	Khóa ngoại liên kết tới tài khoản học viên
semesterId	uuid	Khóa ngoại liên kết tới bảng học kỳ
totalAmount	decimal(15,2)	Tổng số tiền học phí cần đóng
semester	varchar(50)	Tên học kỳ (ví dụ: Kỳ 1, Kỳ 2)
schoolYear	varchar(50)	Năm học (ví dụ: 2025-2026)
content	varchar(255)	Nội dung thu tiền học phí
status	varchar(50)	Trạng thái thanh toán (ví dụ: PAID / UN-PAID)

Bảng 6: Bảng miêu tả chi tiết bảng tuition_fees

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
id	uuid	Khóa chính của yêu cầu đề xuất điểm
userId	uuid	Khóa ngoại liên kết tới học viên gửi đề xuất
subjectResultId	uuid	Khóa ngoại liên kết tới môn học cần sửa
requestType	enum('ADD', 'UPDATE', 'DELETE')	Loại yêu cầu đề xuất điểm
reason	text	Lý do đề xuất chỉnh sửa điểm
proposedLetterGrade	varchar(5)	Điểm chữ đề xuất mới
proposedGradePoint4	double precision	Điểm hệ 4 đề xuất mới
proposedGradePoint10	double precision	Điểm hệ 10 đề xuất mới
attachmentUrl	varchar(500)	Đường dẫn tới tài liệu/hình ảnh minh chứng
status	enum('PENDING', 'APPROVED', 'REJECTED')	Trạng thái phê duyệt đề xuất
reviewerId	uuid	Khóa ngoại liên kết tới cán bộ phê duyệt
reviewNote	text	Ghi chú/lý do phê duyệt hoặc từ chối
reviewedAt	timestamp	Thời gian thực hiện phê duyệt

Bảng 7: Bảng miêu tả chi tiết bảng grade_requests

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Visual Studio Code	https://code.visualstudio.com/
Công cụ soạn thảo văn bản	Overleaf	https://www.overleaf.com/
Công cụ vẽ biểu đồ	Draw.io	https://app.diagrams.net/
Lập trình Server	NodeJS	https://nodejs.org/en/
Xây dựng API	ExpressJS	https://expressjs.com/
Lập trình giao diện	ReactJS	https://react.dev/
Gửi request lên API	Axios	https://axios-http.com/
Hệ quản trị cơ sở dữ liệu	PostgreSQL	https://www.postgresql.org/

Bảng 8: Danh sách thư viện và công cụ sử dụng

0.3.2 Kết quả đạt được

Sau quá trình nghiên cứu và phát triển, đồ án tốt nghiệp đã hoàn thành việc xây dựng hệ thống website Quản lý Học viên với các chức năng chính như sau:

- **Quản lý hồ sơ học viên:** Quản lý thông tin lý lịch cá nhân, quân tịch, cấp bậc, đơn vị và tự động khởi tạo tài khoản đăng nhập tương ứng.
- **Quản lý kết quả học tập & tính toán điểm CPA:** Tự động tính toán điểm trung bình học kỳ, điểm trung bình tích lũy CPA hệ 4 và hệ 10 khi có thay đổi

điểm môn học.

- **Đề xuất cập nhật điểm số:** Cho phép học viên gửi đề xuất sửa đổi/thêm điểm số môn học kèm tài liệu minh chứng (hình ảnh, file PDF); cán bộ chỉ huy có thẩm quyền phê duyệt hoặc từ chối đề xuất.
- **Tự động lập lịch cắt cơm tuần:** Quét thời gian thời khóa biểu học tập của học viên để tự động đánh dấu và cập nhật danh sách cắt cơm theo ngày, tránh lãng phí quân lương.
- **Quản lý lịch học và thời khóa biểu:** Hỗ trợ cán bộ chỉ huy nhập thời khóa biểu học tập của học viên theo học kỳ, xuất lịch học và lịch cắt cơm ra file Excel báo cáo.
- **Quản lý học phí:** Theo dõi chi tiết các đợt đóng học phí, ghi nhận và sửa đổi trạng thái đóng học phí của học viên, phục vụ truy xuất dữ liệu.
- **Phân công lịch trực chỉ huy:** Thiết lập và phân công lịch trực ban của cán bộ chỉ huy đơn vị, ghi nhận ký ca trực và các nhiệm vụ tuần liên quan.
- **Phân quyền người dùng (RBAC) & Bảo mật:** Phân chia rõ ràng 3 vai trò (Học viên, Chỉ huy, Quản trị viên) với cơ chế bảo mật xác thực tài khoản qua JWT và mã hóa mật khẩu bằng bcrypt.

Về mặt đóng gói và triển khai kỹ thuật, mã nguồn ứng dụng được tổ chức khoa học và đóng gói thành hai phân hệ độc lập:

- **Phân hệ Backend API:** Chứa 24 Sequelize Models định nghĩa các thực thể dữ liệu, 26 tập tin định tuyến routes API và 23 Controllers điều khiển. Dung lượng mã nguồn backend xấp xỉ 25 MB (chưa bao gồm thư viện `node_modules`).
- **Phân hệ Frontend App:** Được xây dựng theo mô hình Next.js App Router. Tổ chức giao diện gồm các thư mục `components` (linh kiện dùng chung), `hooks` (các tác vụ lấy dữ liệu), và `store` (quản lý trạng thái). Dung lượng mã nguồn xấp xỉ 15 MB.

0.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

0.4 Kiểm thử

0.4.1 Kiểm thử hộp đen

a, Nhóm chức năng Đăng nhập & Phân quyền

STT	Chức năng	Đầu vào	Đầu ra mong muốn	Kết quả
1	Đăng nhập	Nhập sai thông tin tên tài khoản hoặc mật khẩu	Hiển thị thông báo “Tên đăng nhập hoặc mật khẩu không chính xác”	Đạt
2	Đăng nhập	Bỏ trống tên tài khoản hoặc mật khẩu	Hiển thị thông báo “Vui lòng nhập đầy đủ thông tin”	Đạt
3	Đăng nhập & Phân quyền	Đăng nhập thành công tài khoản Học viên	Chuyển hướng thành công vào trang cá nhân của học viên	Đạt
4	Đăng nhập & Phân quyền	Học viên cố tình truy cập trang cấu hình của Cán bộ	Hệ thống ngăn chặn truy cập và hiển thị thông báo lỗi phân quyền	Đạt

Bảng 9: Kiểm thử nhóm chức năng Đăng nhập & Phân quyền

b, Nhóm chức năng Quản lý hồ sơ học viên

STT	Chức năng	Đầu vào	Đầu ra mong muốn	Kết quả
1	Thêm học viên mới	Nhập trùng mã học viên (student_id đã tồn tại)	Hiển thị thông báo lỗi “Mã học viên đã tồn tại trên hệ thống”	Đạt
2	Thêm học viên mới	Bỏ trống các trường bắt buộc (Mã HV, Họ tên, Lớp học)	Hiển thị cảnh báo yêu cầu nhập đầy đủ thông tin bắt buộc	Đạt
3	Xóa hồ sơ học viên	Cán bộ chỉ huy bấm nút xóa tài khoản học viên	Hồ sơ học viên bị xóa, đồng thời tài khoản người dùng liên kết cũng tự động bị xóa	Đạt
4	Tìm kiếm học viên	Nhập tên học viên hoặc tên lớp vào bộ lọc tìm kiếm	Hiển thị danh sách học viên khớp với từ khóa tìm kiếm	Đạt

Bảng 10: Kiểm thử nhóm chức năng Quản lý hồ sơ học viên

c, Nhóm chức năng Đề xuất và phê duyệt điểm số

STT	Chức năng	Đầu vào	Đầu ra mong muốn	Kết quả
1	Đề xuất điểm số	Học viên tạo yêu cầu chỉnh sửa điểm kèm theo tệp tài liệu minh chứng	Đề xuất được tạo thành công với trạng thái mặc định ban đầu là “PENDING”	Đạt
2	Phê duyệt điểm số	Cán bộ chỉ huy bấm nút chấp nhận đề xuất điểm số	Trạng thái chuyển thành “APPROVED”, điểm số mới được ghi nhận vào bảng điểm	Đạt
3	Tự động tính toán lại CPA	Điểm mới được cập nhật sau khi phê duyệt đề xuất	Điểm trung bình học kỳ, năm học và CPA tích lũy hệ 4/hệ 10 của học viên tự động được tính lại chính xác	Đạt
4	Từ chối đề xuất điểm	Cán bộ chỉ huy bấm nút từ chối đề xuất và nhập lý do phản hồi	Trạng thái chuyển thành “REJECTED”, điểm số cũ của học viên được giữ nguyên	Đạt

Bảng 11: Kiểm thử nhóm chức năng Đề xuất và phê duyệt điểm số

d, Nhóm chức năng Lịch học và tự động cắt cơm

STT	Chức năng	Đầu vào	Đầu ra mong muốn	Kết quả
1	Tự động đồng bộ lịch cơm	Chỉ huy nhập lịch học của học viên trùng vào thời gian ăn/đi lại tại đơn vị	Hệ thống lưu lịch học và tự động tạo bản ghi cắt cơm tương ứng của học viên đó	Đạt
2	Kiểm tra cờ tự động sinh	Kiểm tra trường dữ liệu của lịch cơm vừa sinh tự động	Bản ghi cắt cơm lưu thuộc tính <code>isAutoGenerated = true</code>	Đạt
3	Điều chỉnh lịch cơm	Học viên thực hiện điều chỉnh hủy cắt cơm thủ công	Trạng thái cập nhật thành công, thuộc tính chuyển đổi thành <code>isAutoGenerated = false</code>	Đạt

Bảng 12: Kiểm thử nhóm chức năng Lịch học và tự động cắt cơm

e, Nhóm chức năng Quản lý học phí

STT	Chức năng	Đầu vào	Đầu ra mong muốn	Kết quả
1	Cập nhật học phí	Chỉ huy cập nhật trạng thái đóng học phí sang “Đã đóng”	Trạng thái học phí cập nhật thành “PAID”, học viên có thể xem trực tuyến ngay lập tức	Đạt
2	Xuất báo cáo học phí	Chỉ huy nhấn nút kết xuất báo cáo học phí của một lớp	Hệ thống xuất thành công tệp tin báo cáo định dạng Excel/Word đúng biểu mẫu	Đạt
3	Tra cứu học phí	Học viên truy cập mục kiểm tra học phí cá nhân	Hiển thị chính xác các đợt học phí và trạng thái đóng/nợ tương ứng	Đạt

Bảng 13: Kiểm thử nhóm chức năng Quản lý học phí

0.4.2 Kiểm thử tương thích

STT	Thiết bị	Phần cứng	Phần mềm	Kết quả
1	Laptop ASUS TUF GAMING F15	Màn hình: 15.6 inch, CPU: 11th Gen Intel Core i5 11400H, RAM: 16 GB	HĐH: Windows 11; Trình duyệt: Google Chrome	Đạt
2	ACER NITRO 5	Màn hình: 15.6 inch, CPU: 12th Intel Core i5-12500H, RAM: 16 GB	HĐH: Windows 11; Trình duyệt: Google Chrome	Đạt
3	Laptop MSI Modern 14 C12MO-660VN	Màn hình: 15.6 inch FHD, CPU: AMD Ryzen 7-5700U, RAM: 16 GB	HĐH: Windows 11; Trình duyệt: Google Chrome	Đạt

Bảng 14: Kiểm thử thiết bị và môi trường

0.5 Triển khai

Hệ thống được thiết kế để có thể triển khai linh hoạt trên hạ tầng đám mây ảo hoặc máy chủ vật lý chuyên dụng của đơn vị đào tạo.

Kiến trúc triển khai vật lý:

- **Hệ điều hành máy chủ:** Ubuntu Server 22.04 LTS hoặc Linux CentOS 8.
- **Cấu hình khuyến nghị:** CPU 4 Cores, RAM 8 GB, SSD 100 GB.
- **Môi trường thực thi:** Node.js v20 runtime cho API Server, PostgreSQL v16 Engine cho CSDL, MinIO Server cho việc lưu trữ Object Storage đính kèm.

Triển khai container hóa bằng Docker Compose: Để đơn giản hóa quy trình cài đặt và đảm bảo tính đồng nhất môi trường giữa phát triển và vận hành thực tế, dự án tích hợp sẵn tập tin cấu hình đóng gói `docker-compose.yml`.

Mô tả hoạt động của các phân hệ Docker dịch vụ:

1. `database-postgres`: Khởi chạy PostgreSQL, ánh xạ cổng 5432, cấu hình volume `pgdata` để lưu trữ dữ liệu bền vững.
2. `object-storage-minio`: Khởi chạy máy chủ MinIO lưu trữ tệp tin, expose cổng điều khiển giao diện web Console ở cổng 9001 và cổng API lưu trữ ở 9000.
3. `backend-api-server`: Đóng gói mã nguồn backend Express.js, cấu hình liên kết mạng phụ thuộc vào database và storage. Lắng nghe yêu cầu HTTP ở cổng 6868.
4. `frontend-client-app`: Đóng gói ứng dụng Next.js, ánh xạ cổng hiển thị giao diện chính ra cổng 3000 của máy chủ vật lý để người dùng truy cập.

Với thiết lập này, quản trị viên chỉ cần cài đặt Docker/Docker Compose trên máy chủ vật lý và thực hiện lệnh duy nhất `docker-compose up -d --build` để khởi chạy toàn bộ hệ thống quản lý học viên.

0.5.1 Kết quả triển khai thử nghiệm

Hệ thống đã được triển khai thử nghiệm với kết quả khả quan. Về độ tin cậy, hệ thống hoạt động ổn định trong suốt thời gian thử nghiệm, ít xảy ra lỗi hoặc gián đoạn dịch vụ, đảm bảo thời gian hoạt động liên tục (uptime) đạt trên 99%. Về tương tác người dùng, cán bộ chỉ huy và học viên tương tác tốt với các tính năng cốt lõi, đặc biệt là tính năng tự động tính lịch cắt cơm và đề xuất cập nhật điểm trực tuyến, giúp giảm thiểu 80% thủ tục giấy tờ thủ công tại đơn vị. Về phản hồi người dùng, đa số cán bộ và học viên đều đánh giá tích cực về tính năng tự động hóa và giao diện trực quan, cho thấy hệ thống đáp ứng tốt nhu cầu thực tế của công tác quản lý đào tạo. Một số vấn đề nhỏ đã được ghi nhận như thời gian khởi động nguội của các Docker container và độ trễ khi tải ban đầu danh sách học viên quy mô lớn; những điểm hạn chế này sẽ được nghiên cứu và tối ưu hóa trong các phiên bản nâng cấp tiếp theo.